

Cisco Packet Tracer Helper — Weekend Project Plan

An MCP server that helps design and configure network labs for Packet Tracer

1. What We're Building

Cisco Packet Tracer is the simulator students use to practice building networks (routers, switches, PCs, cables) for CCNA-style courses. It has no way for outside programs to control it directly — there's no API and no webhook. So instead of trying to control Packet Tracer itself, we build a helper tool that does the thinking for the student, and the student does the clicking and typing inside Packet Tracer.

This helper will be an **MCP server** — a small program that plugs into an AI assistant (like Claude) and gives it new abilities. Once connected, you can just ask the assistant things like *"Design a network with 3 routers and 2 switches using OSPF"* and it will use our tools to build a full answer.

2. Why This Fits in a Weekend

This project is mostly about writing clear logic and templates in Python — not about fighting difficult software. There's no login system, no external servers to talk to, and no waiting on slow APIs. That's what makes it realistic to finish in two days.

- No API keys or accounts needed — everything runs on your own computer.
- No internet calls required while it runs — it's all local calculations and templates.
- You can ship a working version with just 2 of the 4 tools and still have something useful.

3. The Four Tools We'll Build

Tool	What it does	In plain words
design_topology	Takes your requirements (e.g. "3 routers, 2 switches, OSPF") and works out the device list, how they connect, and the network addresses.	Starts the network on paper before you build it.
generate_configs	Turns the plan into real Cisco IOS commands you can copy and paste into each device inside Packet Tracer.	Writes the tasks to each device you told it to do.
render_diagram	Draws a simple picture of the topology (which device connects to which) so you know what to place on the canvas.	Creates a map of what to build.
verify_config	You paste back the config you typed into Packet Tracer, and it checks that against the plan and config settings.	A safety check that you didn't skip a step or make a typo.

Important limit to know: Packet Tracer's file format (.pkt) is private and undocumented, so none of these tools can automatically build the topology inside Packet Tracer for you. You'll still drag devices onto the canvas and type the configs in yourself — this tool just makes sure you know exactly what to place and what to type, and checks your work after.

4. Weekend Schedule

Saturday — Core Logic (about 5–6 hours)

Task	Time	Details
Set up the project	30 min	Create the basic MCP server skeleton in Python using the official MCP SDK. Mostly copy-paste.
Build design_topology	2 hrs	Write the Python logic that takes a request and outputs a device list + IP address plan (this is the core logic).
Build generate_configs	2.5 hrs	Write text templates for common device types (router, switch) and routing methods (static routing, OSPF, etc.).

Sunday — Polish + Extra Tools (about 4–6 hours)

Task	Time	Details
Build render_diagram	1.5 hrs	Generate a simple diagram (boxes and lines) showing the topology layout.
Build verify_config	2.5 hrs	Write a checker that reads pasted-in config text and compares it against the expected plan, reporting errors.
Test everything together	1.5 hrs	Connect the finished MCP server to Claude Desktop (or another client) and run through a few test cases.

5. How to Keep It Simple (So You Actually Finish)

- Only support the basics at first: static routing, OSPF, VLANs, and simple switching. Skip BGP, NAT, and ACLs for version 1 — add them later if you want.
- For verify_config, just check the important settings (IP addresses, hostnames, routing protocol) instead of trying to catch every possible typo.
- If you're short on time Sunday, ship with just design_topology and generate_configs — that alone is already useful.

6. What You'll Need Before Starting

- Python installed on your computer (3.10 or newer).
- The official MCP Python SDK (installed with pip).
- Claude Desktop (or another MCP-compatible client) to test the server as you build it.
- Basic familiarity with subnetting and Cisco IOS commands, since you're writing the logic that generates them.

7. What You'll Have by Sunday Night

A working local tool where you can describe a network in plain English, and get back a device list, an IP address plan, ready-to-paste Cisco commands, and a diagram — all generated instantly, so you can spend your time actually building and learning inside Packet Tracer instead of looking things up.